

Stage 1 Lab - Human vs AI: Building Your First SmartCare Prototype

Learning objectives

- Create and run a simple Python file with basic input,output and processing statements
- Use lists, dictionaries and functions to enhance the Python file
- Build a small SmartCare appointment prototype.
- Use AI as a tutor rather than a replacement.
- Compare human-written and AI-generated code.
- Verify AI-generated code through execution and test inputs.
- Document a short AI-use reflection.

Files to create and commit in GitHub

stage01/
smartcare_v01.py
comparison.md
reflection.md
ai_usage.md

Part A - Understand the Problem: AI OFF

SmartCare needs a small prototype that allows a receptionist to record patient appointments. Each appointment records patient name, practitioner name and appointment time.

What data must be stored?

- Name of patients
- Name of practitioner
- Appointment date and time.

What functions might be useful?

- Add booking appointments
- having to view the appointments
- Edit an appointment when necessary
- Search for user profile

What could go wrong?

- Incorrect data input
- Appointing a date or time which is not valid

- Missing appointment information
- Mistakenly deleting appointment

What requirements are unclear?

- Who can edit or delete appointments?
- Are multiple bookings available?
- Can bookings be done in advance?
- Should the system prevent double-booking?
- Who can edit the appointments?

Part B - Build a Human-Written Prototype: AI OFF

#task 1

Create and run a simple Python file with basic input,output statements

```
print("Welcome to SmartCare: Community Clinic Appointment Booking System!")
```

First Appointment

```
patient1_name = 'Alice Smith'
```

```
practitioner1_name = 'Dr. John Doe'
```

```
appointment1_time = '2024-07-20 10:00 AM'
```

```
print(f"Patient: {patient1_name} | Practitioner: {practitioner1_name} | Time: {appointment1_time}")
```

Second Appointment

```
patient2_name = 'Bob Johnson'
```

```
practitioner2_name = 'Dr. Jane Roe'
```

```
appointment2_time = '2024-07-20 11:30 AM'
```

```
print(f"Patient: {patient2_name} | Practitioner: {practitioner2_name} | Time: {appointment2_time}")
```

.....

#task1enhanced

Use lists, dictionaries and functions to enhance the Python file

```
appointments = []
```

```
def book_appointment(patient_name, practitioner_name, appointment_time):
```

```
    if not patient_name:
```

```
        raise ValueError("Patient name cannot be empty")
```

```
    appointment = {
```

```
        "patient": patient_name,
```

```
        "practitioner": practitioner_name,
```

```
        "time": appointment_time
```

```

    }
    appointments.append(appointment)

def display_appointments():
    if not appointments:
        print("No appointments recorded.")
        return
    for appointment in appointments:
        print(f"Patient: {appointment['patient']} | Practitioner:
{appointment['practitioner']} | Time: {appointment['time']}")

print("Welcome to SmartCare: The Clinical Appointment Booking System!")
book_appointment('Alice Smith', 'Dr. John Doe', '2024-07-20 10:00 AM')
book_appointment('Bob Johnson', 'Dr. Jane Roe', '2024-07-20 11:30 AM')
display_appointments()

```

^ Now, run both programs , and identify at least five limitations.

Part C - Use AI as Tutor: AI ON (Use only UC approved GenAI Tool such as Microsoft CoPilot)

Suggested prompt structure:

Act as a Python tutor.

I am learning introductory software technology.

Here is a small appointment-booking function.

1. Explain what the code does.

It prints a welcome message, stores appointment details, and displays them. The enhanced version uses a list and dictionaries to store multiple appointments and functions to organise the logic.

Expanded explanation

- The first part simply defines three variables for each appointment and prints them.
- The enhanced part:
 - Creates an empty list called **appointments**.
 - Defines **book_appointment**, which:
 - Validates the patient name.
 - Builds a dictionary containing patient, practitioner, and time.
 - Appends that dictionary to the appointments list.
 - Defines **display_appointments**, which:
 - Checks if the list is empty.
 - Loops through each appointment and prints its details.
 - Calls the booking function twice, then displays all stored appointments.

2. Identify three limitations.

- **No user input** — all appointments are hard-coded, so the user cannot enter their own details.
- **No data validation** beyond checking the patient name. Invalid dates, empty practitioner names, or duplicate bookings are not handled.
- **No persistence** — once the program ends, all appointments disappear because nothing is saved to a file or database.

3. Suggest improvements.

- Add **input handling** so users can type appointment details.
- Add **error checking** for practitioner names, date formats, and time clashes.
- Add **file storage** (e.g., JSON or CSV) so appointments persist between runs.
- Add a **menu system** to let users choose actions like “Book appointment” or “View appointments”.

4. Do not rewrite the whole application.

5. Ask me two questions to test my understanding.

- Why is a **list of dictionaries** a better structure for storing multiple appointments than using separate variables like `patient1_name`, `patient2_name`, etc.
- What would happen if you removed the `if not patient_name: check inside book_appointment?`

Part D - Generate an Alternative: AI ON

Ask AI to create a simple beginner-friendly Python function that stores patient name, practitioner name and appointment time. Explicitly prohibit a database or GUI.

```
def create_appointment(patient_name, practitioner_name, appointment_time):
```

```
    if not patient_name or not practitioner_name or not appointment_time:
```

```
        raise ValueError("All fields must be provided.")
```

```
    appointment = {
```

```
        "patient": patient_name,
```

```
        "practitioner": practitioner_name,
```

```
        "time": appointment_time
```

```
    }
```

return appointment

Example usage

```
appt = create_appointment("Alice Smith", "Dr. John Doe", "2024-07-20 10:00 AM")
```

```
print(appt)
```

Part E - Compare Human and AI Versions

Question	Human version	AI version
Easy to understand?	yes	yes
Runs successfully?	yes	yes
Uses only required features?	yes	yes
Adds assumptions?	no	yes
Handles errors?	no	yes
Could I explain it?	yes	yes

Part F - Verify Behaviour

- Normal appointment- done okay
- Blank patient name- not handling properly
- Two appointments for the same practitioner/time- not handling properly
- Strange input such as patient_name=None or appointment_time=None- not handling properly

Part G - Improve One Thing

Choose exactly one controlled improvement, for example: if not patient_name: raise ValueError("Patient name cannot be empty")

Error handling should be improved to improve efficiency and reducing time waste.

Part H - Reflection (150-250 words)

What did you build before using AI?

- I build a basic appointment system where you can list the name of patients and set time

What did AI help you understand?

- It helped to understand my mistakes, improve the code and make things simpler

Did AI make assumptions?

- Yes, it made assumptions for error handling and improvisation

How did you verify the AI output?

- I used vs code

What engineering work remained for you?

- Code function and its reliability

Submission checklist [GitHub Commit]

- Python file runs.
- Comparison table completed.
- Normal and unusual inputs tested.
- AI assistance documented.
- Reflection completed.
- I can explain my code.